FOOD DELIVERY SYSTEM

Submitted in partial fulfillment of the requirements.

of the degree of

Bachelor of Technology

in

Artificial Intelligence & Machine Learning

By

Sagar Mudaliyar 125A9027

Lavish Mudaliar 125A9026

Atharv Patil 125A9040

Under the Guidance of

Prof. Sumedha A. Maindalkar



Department of Artificial Intelligence & Machine Learning

SIES GRADUATE SCHOOL OF TECHNOLOGY

Nerul, Navi Mumbai 400706

University of Mumbai

2025-2026

CERTIFICATE

This is to certify that the project entitled “Food Delivery System – Database Management System” has been carried out as an academic project by the student/project team. The project presents the design and implementation of a web-based food delivery application supported by a relational database.The project demonstrates user authentication, restaurant management, food item management, cart and order concepts, delivery assignment, relational database design, and CRUD operations. The database design and relationships are represented through the supplied Food Delivery System ER diagram.

ATHARV PATIL – 125A9040

SAGAR MUDALIYAR – 125A9027

LAVISH MUDALIAR – 125A9026

Prof . Sumedha A. Maindalkar	Mrs. Kalyani Pampattiwar	Dr.K.Sudha Lakshmi
Guide	Head of Department	Principal

DECLARATION

I hereby declare that the project report entitled “Food Delivery System – Database Management System” is prepared for academic purposes and presents the work carried out for the DBMS project .I declare that this written submission represents my ideas in my own words and where others’ ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

ATHARV PATIL – 125A9040 -
SAGAR MUDALIYAR – 125A9027 -
LAVISH MUDALIAR – 125A9026 –
SIGNAURE

ACKNOWLEDGEMENT

I express my sincere gratitude to the principal Dr. K. Lakshmi Sudha , head of the department Mrs.Kalyani Pampattiwar , project guide Prof. Sumedha A. Maindalkar for her guidance, help and useful suggestion which helped in completing our project work in time. Also, we would like to thank the entire faculty of the Artificial Intelligence and Machine Learning department for their valuable ideas and timely assistance in this project, last but not least, we would like to thank our teaching and non-teaching staff members of our college for their support, in facilitating timely completion of this project.

.

Project Team

Sagar Mudaliyar

Lavish Mudaliar

Atharva Patil

ABSTRACT

The Food Delivery System is a web-based application designed to manage the main activities involved in an online food ordering and delivery platform. The system provides a structured database for users, restaurants, food items, carts, orders, order items, and delivery personnel.

The project uses a relational database approach in which primary keys and relationships connect the major entities. A user can place multiple orders and maintain a cart. An order contains order items, while each order item refers to a food item. Restaurants offer food items, and delivery personnel can be assigned to orders. These relationships are represented in the ER diagram supplied for the project.

The application demonstrates database operations through a web interface. CRUD operations are used for managing restaurant and food-item records, while authentication provides a login-based entry point. The system is intended to provide a practical demonstration of how a DBMS can support a real-world food ordering workflow.

The project therefore provides a foundation that can be expanded with additional features such as online payment integration, order tracking, ratings and reviews, offers, notifications, and more advanced administrative controls.

CONTENTS

SR. NO.	CHAPTER	PAGE NO.
1	INTRODUCTION	7
2	SYSTEM REQIUREMENTS: S/W used	8
3	SYSTEM DESIGN: ER-DIAGRAM	10
4	IMPLEMENTATION	12
5	CODE SNIPPETS (DAX)	23
6	RESULT	30
7	CONCLUSION	35
8	REFERENCES	37

Chapter 1: Introduction

1.1 Introduction and Need of the Project

Online food delivery systems manage several related types of information at the same time. User accounts, restaurants, food items, carts, orders, order items, and delivery personnel are connected to one another. Maintaining these records in an organized relational database makes it easier to store, retrieve, update, and delete information.

The Food Delivery System developed in this project provides a centralized web-based environment for food ordering and delivery management. Instead of treating each record as an isolated item, the system uses related entities connected through primary keys and relationships.

The project also demonstrates how a web application can communicate with a database through a backend framework. The login system provides an entry point for users, while CRUD operations provide practical database interaction for restaurant and food-item management.

1.2 Problem Statement in Broad Sense

Traditional food ordering processes can involve separate records for customers, restaurants, menus, orders, and delivery details. Without a structured system, maintaining these records can become difficult, especially when an order contains multiple food items and each food item belongs to a restaurant.

The project therefore addresses the problem of developing a database-backed food delivery application that can manage users, restaurants, food items, carts, orders, order items, and delivery personnel through a unified system

1.3 Objectives

- Develop a web-based Food Delivery System using a relational database.
- Create a login/authentication interface for users.
- Design an ER model containing the major food delivery entities.
- Use primary keys and relationships to maintain data integrity.
- Implement CRUD operations for restaurant and food-item records.
- Manage cart and order information through related database entities.
- Represent order items separately so that an order can contain multiple food items.
- Represent delivery personnel and their assignment to orders.
- Connect the frontend/application layer with the PostgreSQL database.
- Test database operations through the web interface.

.

1.4 Scope of the Project

The project scope includes user login, restaurant management, food-item management, cart and order database design, order-item management, delivery-person assignment, and PostgreSQL-based relational storage. The implemented web interface can be extended with additional modules without changing the overall database concept.

The current Student Management implementation demonstrates the complete CRUD cycle. It is connected to the existing students table and uses the existing REST API endpoints. The Admin Dashboard provides the appropriate location for administrative student record management, while the Student Dashboard can remain focused on student-specific services.

•
•

1.5 Organization of the Report

Chapter 1 introduces the project, its need, problem statement, objectives, and scope. Chapter 2 describes the software and hardware requirements. Chapter 3 presents the ER diagram and explains its entities and relationships. Chapter 4 describes implementation and CRUD operations. Chapter 5 presents important code snippets. Chapter 6 documents results and testing. Chapter 7 presents the conclusion, challenges, learning outcomes, and future scope. Chapter 8 provides references.

Chapter 2: System Requirements (Software Used)

2.1 Microsoft Windows Operating System

The project can be developed and tested in a Windows environment using a code editor, Python/Django development environment, PostgreSQL, and a modern web browser.

- Windows 10 or later
- 64-bit system recommended
- PostgreSQL database service
- Python and Django installed
- Modern web browser

2.2 Visual Studio Code

Visual Studio Code can be used as the main source-code editor for the Django project. It provides project folder management, Python/HTML/CSS editing, integrated terminal access, and error identification.

2.3 Python and Django

Python is used as the programming language and Django is used as the web application framework. Django provides URL routing, views, templates, forms, authentication support, and database integration.

2.4 PostgreSQL

PostgreSQL is used as the relational database management system. It stores the structured data represented by the Food Delivery System ER diagram. Primary keys, foreign keys, and other constraints can be used to maintain relationships and data integrity.

2.5 Development Environment

Component	**Project Use**
Operating System	Windows
Programming Language	Python
Web Framework	Django
Database	PostgreSQL
Editor	Visual Studio Code
Browser	Google Chrome / modern browser
Data Model	Relational database

•

2.6 Hardware Requirements

- Computer capable of running Windows, Python, Django, PostgreSQL, and a modern browser.
- At least 4 GB RAM is recommended; more RAM is useful when running the editor, browser, database, and development server together.
- Adequate storage for the project files and PostgreSQL data.

Keyboard, mouse, and display suitable for web application development

- Adequate disk space for the project source code, dependencies, and PostgreSQL data.
- Keyboard, mouse, and display suitable for web application development.

Chapter 3: System Design – ER Diagram

3.1 ER Diagram Overview

The Entity-Relationship (ER) diagram is the database blueprint of the Food Delivery System. It identifies the major entities, their attributes, relationships, primary keys, and cardinalities. The supplied diagram contains User, Cart, Order, OrderItem, FoodItem, Restaurant, and DeliveryPerson as the main entities

3.2 ER Diagram

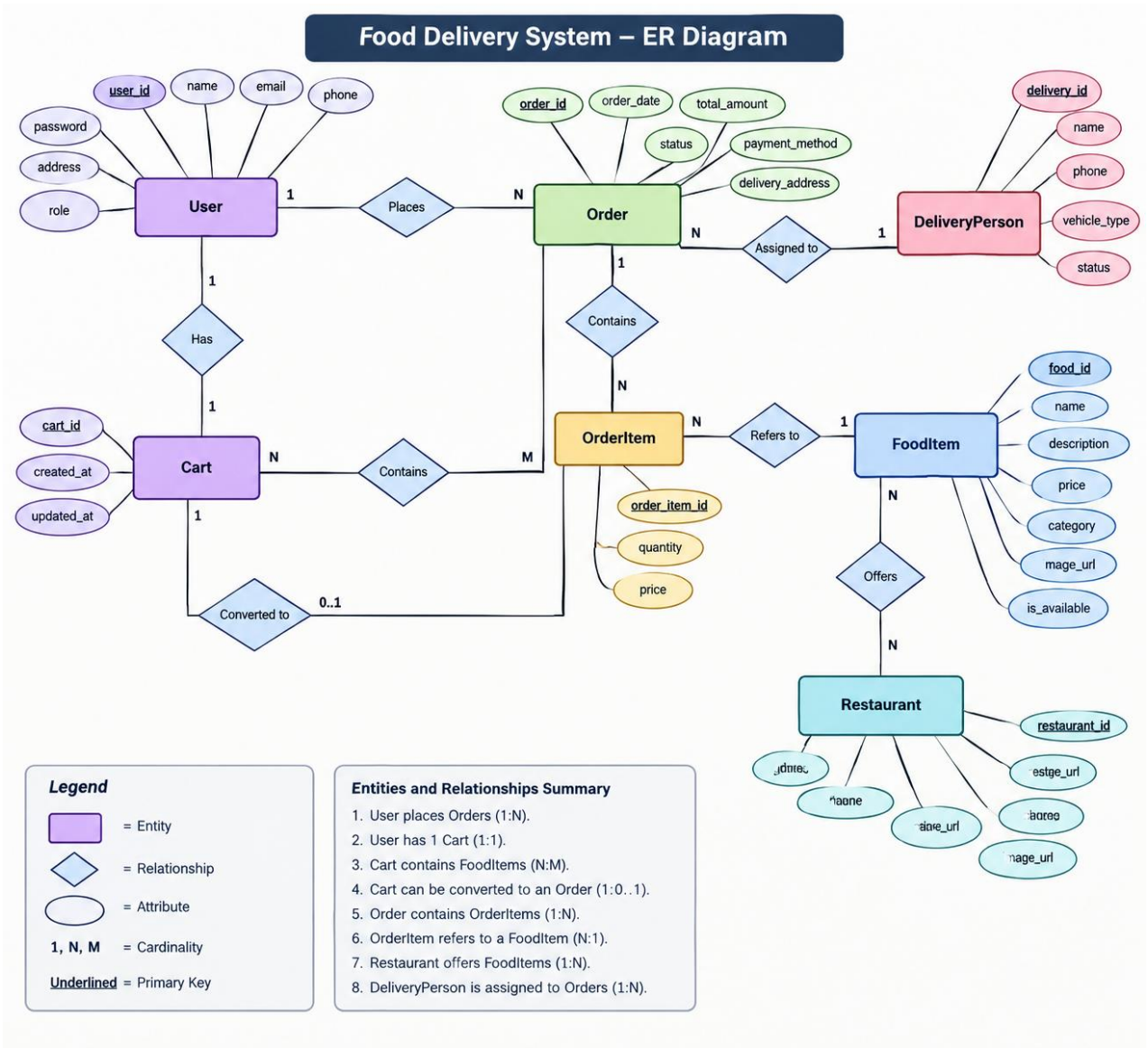


Figure 3.1: College Management System – ER Diagram

3.3 User Entity

The User entity stores user account information. Its attributes in the supplied ER diagram are user_id, name, email, phone, password, address, and role. user_id is the primary key.

3.4 Cart Entity

The Cart entity represents the user's shopping cart. The diagram contains cart_id, created_at, and updated_at. The relationship with User is shown as one-to-one, meaning the model associates one cart with a user.

3.5 Order Entity

The Order entity represents a placed food order. Its attributes are order_id, order_date, total_amount, status, payment_method, and delivery_address. order_id is the primary key. A User can place multiple Orders.

3.6 OrderItem Entity

OrderItem represents an individual food entry within an order. The supplied diagram contains order_item_id, quantity, and price. An Order contains multiple OrderItems, and each OrderItem refers to one FoodItem.

3.7 FoodItem Entity

FoodItem represents a food product available for ordering. The diagram contains food_id, name, description, price, category, image_url, and is_available. food_id is the primary key.

3.8 Restaurant Entity

Restaurant represents the restaurant that offers food items. The supplied diagram shows restaurant_id along with name, address, phone, image_url, and other restaurant-related attributes shown in the diagram. restaurant_id is the primary key.

3.9 DeliveryPerson Entity

DeliveryPerson stores information about delivery personnel. The diagram contains delivery_id, name, phone, vehicle_type, and status. delivery_id is the primary key.

3.10 Relationships and Cardinality

Relationship	Cardinality shown in ER diagram	Meaning
User – Places – Order	1 : N	One user can place many orders.
User – Has – Cart	1 : 1	A user is associated with one cart.
Cart – Contains – OrderItem	N : M	The diagram models cart contents as a many-to-many relationship.
Cart – Converted to – Order	1 : 0..1	A cart may be converted to an order.
Order – Contains – OrderItem	1 : N	One order contains multiple order items.
OrderItem – Refers to – FoodItem	N : 1	Many order items can refer to one food item.
Restaurant – Offers – FoodItem	N : N	The diagram shows an N:N offers relationship.
Order – Assigned to – DeliveryPerson	N : 1	Many orders can be assigned to one delivery person.

3.11 Importance of the ER Diagram

The ER diagram helps convert the requirements of the food delivery application into a structured relational model. It shows which information belongs to each entity and how entities are connected. This provides a basis for creating database tables and writing SQL queries involving users, carts, orders, food items, restaurants, and delivery personnel.

Chapter 4: Implementation

4.1 Project Folder Structure

4.1.1 Main project structure

The project can be organized as a Django project containing the main project configuration, application modules, templates, static files, database configuration, and migration files.

```
fooddelivery/
├──manage.py
├──fooddelivery/
├──settings.py
├──urls.py
│
└──accounts/
    ├──models.py
    ├──views.py
    ├──urls.py
    ├──templates/
    └── migrations/
```

4.2 Database Creation and Tables

The PostgreSQL database stores the entities represented in the ER diagram. Django models can be used to map application objects to database tables. The model definitions should preserve the primary-key and relationship structure of the ER diagram.

4.2.1 User Table

The User entity stores information about users who access the Food Delivery System.

The user account contains authentication and personal information required for login and application access.

The major attributes include:

- user_id
- name
- email
- phone
- password
- address
- role

4.2.2 Restaurant Table

The Restaurant entity stores information about restaurants available on the Food Delivery System.

The restaurant information includes:

- restaurant_id

- name
- description
- cuisine
- rating
- delivery_time
- category

4.3 Food item table

- food_id
- name
- description
- price
- category
- image_url
- is_available

4.2.4 Cart Table

The Cart entity represents the shopping cart associated with a user.

The major attributes include:

- cart_id
- created_at
- updated_at

4.2.5 CartItem Table

The CartItem structure allows the system to store information such as:

- Cart
- Food item
- Quantity

4.2.6 Order Table

The Order entity represents a completed food order.

The major attributes represented in the ER diagram are:

- order_id
- order_date
- total_amount
- status
- payment_method
- delivery_address

User (1) → Order (N)

4.2.7 OrderItem Table

The OrderItem entity represents individual food items included in an order.

The major attributes are:

- order_item_id
- quantity
- price

An Order can contain multiple OrderItems.

Each OrderItem refers to a FoodItem.

Therefore, the relationship is:

Order (1) → OrderItem (N)

and

FoodItem (1) → OrderItem (N)

4.2.8 DeliveryPerson Table

The DeliveryPerson entity stores information about delivery personnel.

The attributes represented in the ER diagram include:

- `delivery_id`
- `name`
- `phone`
- `vehicle_type`
- `status`

4.3 Backend Server Setup

The backend of the FoodGo application is developed using **Python and Django**.

Django manages:

- URL routing
- Views
- Forms
- Authentication
- Database communication
- CRUD operations
- Templates
- User sessions

4.3.1 Django and Database Configuration

The Django project contains a main project directory and an accounts application.

The application is registered in INSTALLED_APPS.

The database configuration connects Django with PostgreSQL.

A representative configuration is:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'food_delivery',
        'USER': 'postgres',
        'PASSWORD': 'your_password',
        'HOST': 'localhost'
        'PORT': '5432',
    }
}
```

4.4 Backend Development Server

The Django development server is used to run and test the FoodGo application locally.

The server is started using:

python manage.py runserver

After starting the server, the application can be accessed through the browser at:

http://127.0.0.1:8000/

The Django server processes browser requests and communicates with PostgreSQL through Django's ORM.

4.4.1 Application URL Routing

The main project URL configuration connects the FoodGo application with the Django project.

A simplified URL configuration is:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('accounts.urls')),
]
```

.

4.5 Login and Authentication Implementation

The FoodGo system uses Django's built-in authentication functionality.

The login system allows users to access the application using their account credentials.

The project supports:

- User registration
- User login
- User logout
- Protected homepage
- Username/email login
- Invalid login handling

The login page provides fields for the user's username/email and password.

After successful authentication, the user is redirected to the FoodGo home page.

4.5.1 User Registration

The registration page allows a new user to create an account.

The registration form contains:

- Full Name
- Email
- Username
- Password
- Confirm Password

.4.5.2 Login Implementation

The login view validates the submitted credentials.

A simplified representation of the login logic is:

```
if username_or_email and password:
    user = authenticate(
        request,
        username=username_or_email,
```

```
        password=password
    )

    if user is not None:
        login(request, user)
        return redirect('home')
```

4.5.3 Logout Implementation

The logout functionality terminates the authenticated session.

A simplified implementation is:

```
from django.contrib.auth import logout

def logout_view(request):
    logout(request)
    return redirect('login')
```

4.6 Restaurant CRUD Backend

Restaurant Management is implemented using the four fundamental database operations:

- Create
- Read
- Update
- Delete

4.6.1 Create Restaurant

The Create operation allows a new restaurant to be added to the database.

A simplified Django implementation is:

```
def restaurant_create_view(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        description = request.POST.get('description')
        cuisine = request.POST.get('cuisine')
        rating = request.POST.get('rating')
        delivery_time = request.POST.get('delivery_time')
        category = request.POST.get('category')

        Restaurant.objects.create(
            name=name,
            description=description,
            cuisine=cuisine,
            rating=rating,
            delivery_time=delivery_time,
            category=category
        )

        return redirect('home')
```

The submitted restaurant details are stored in PostgreSQL through Django ORM.

4.6.2 Read Restaurants

The Read operation retrieves restaurant records from the database.

A simplified implementation is:

```
restaurants = Restaurant.objects.all()
```

The retrieved restaurants are then passed to the template:

```
return render(
    request,
    'accounts/home.html',
    {'restaurants': restaurants}
)
```

The restaurant information can then be displayed on the FoodGo homepage.

4.6.3 Update Restaurant

The Update operation allows an existing restaurant record to be modified.

A simplified implementation is:

```
restaurant = get_object_or_404(
    Restaurant,
    id=restaurant_id
)

if request.method == 'POST':
    restaurant.name = request.POST.get('name')
    restaurant.description = request.POST.get('description')
    restaurant.cuisine = request.POST.get('cuisine')
    restaurant.rating = request.POST.get('rating')
    restaurant.delivery_time = request.POST.get('delivery_time')
    restaurant.category = request.POST.get('category')

    restaurant.save()

    return redirect('home')
```

The updated information is saved back into the PostgreSQL database.

4.6.4 Delete Restaurant

The Delete operation removes a selected restaurant from the database.

A simplified implementation is:

```
restaurant = get_object_or_404(
```

```
    Restaurant,
    id=restaurant_id
)

restaurant.delete()

return redirect('home')
```

4.7 Food Item CRUD Backend

Food Item Management is another major CRUD module in the FoodGo system.

It allows food items to be:

- Created
- Read
- Updated
- Deleted

This provides database interaction through the FoodGo web interface.

4.7.1 Create Food Item

A new food item can be added through the **Add Food Item** interface.

A simplified implementation is:

```
def food_create_view(request):
    if request.method == 'POST':
        FoodItem.objects.create(
            name=request.POST.get('name'),
            description=request.POST.get('description'),
            price=request.POST.get('price'),
            category=request.POST.get('category'),
            image_url=request.POST.get('image_url'),
            is_available=True
        )

        return redirect('foods')
```

.4.7.2 Read Food Items

The Read operation retrieves food items from the database.

```
food_items = FoodItem.objects.all()
```

The food items are displayed through the FoodGo food listing page.

Examples of food records used in the project include:

Food Item	Price
Farmhouse Pizza	₹349

Food Item	Price
Margherita Pizza	₹299
Paneer Tikka Pizza	₹379

4.7.3 Update Food Item

The Update operation allows existing food information to be edited.

For example:

```
food_item = get_object_or_404(
    FoodItem,
    id=food_item_id
)

food_item.name = request.POST.get('name')
food_item.description = request.POST.get('description')
food_item.price = request.POST.get('price')
food_item.category = request.POST.get('category')

food_item.save()
```

4.7.4 Delete Food Item

The Delete operation removes a selected food item.

```
food_item = get_object_or_404(
    FoodItem,
    id=food_item_id
)

food_item.delete()

return redirect('foods')
```

After deletion, the selected food item is removed from the food list.

4.8 Cart and Order Integration

The FoodGo database design includes Cart, CartItem, Order, and OrderItem entities.

The cart allows users to maintain food items before completing an order.

The general workflow is:

```
User
  ↓
Browse Food Items
  ↓
Add Food Item
  ↓
Cart
  ↓
```

```
Order
  ↓
Order Items
  ↓
Delivery
```

The Cart entity is associated with a User.

CartItem connects the Cart with FoodItem.

When the user proceeds with an order, the selected food items can be represented through OrderItem records.

4.8.1 Cart Management

The cart stores the food items selected by the user.

The system can maintain:

- Selected food item
- Quantity
- Cart information
- Created time
- Updated time

The Cart and CartItem entities support the database structure required for the shopping-cart functionality.

4.8.2 Order Management

The Order entity stores the information about a placed order.

The order contains:

- Order ID
- Order date
- Total amount
- Status
- Payment method
- Delivery address

An Order can contain multiple OrderItems.

4.8.3 Order Item Management

OrderItem represents the individual food items belonging to an order.

For example:

```
Order #101
│
├── Farmhouse Pizza × 1
```

```
├── Margherita Pizza × 2
└── Paneer Tikka Pizza × 1
```

The OrderItem table stores the quantity and price associated with the selected food item.

4.9 Delivery Person Integration

The FoodGo database design includes a DeliveryPerson entity.

The DeliveryPerson table stores:

- Delivery ID
- Name
- Phone
- Vehicle Type
- Status

The Order entity is connected to DeliveryPerson using the **Assigned to** relationship.

The relationship allows orders to be associated with delivery personnel.

4.9.1 Delivery Assignment

The relationship represented in the ER diagram is:

Order (N) → Assigned to → DeliveryPerson (1)

This means multiple orders can be associated with a delivery person according to the database design.

The delivery person's status can be used to represent their current delivery availability/status.

4.10 FoodGo Homepage Integration

The FoodGo homepage provides the main interface for the food delivery application.

The homepage includes food-delivery features such as:

- Navigation bar
- Location display
- Food categories
- Restaurant information
- Food items
- Login/logout access
- Food browsing

The design uses a modern food-delivery theme with warm orange and teal interface elements.

The homepage is connected to the backend so that restaurant and food-item information can be displayed from the database.

4.10.1 Food Categories

The FoodGo system includes categories such as:

- Pizza
- Burgers
- Indian
- Chinese
- Desserts
- Healthy
- Beverages

These categories help organize food items for the user interface.

4.11 Database Migration

Django migrations are used to apply model changes to the PostgreSQL database.

The following commands are used:

```
python manage.py makemigrations
```

followed by:

```
python manage.py migrate
```

Chapter 5: Code Snippets

The FoodGo project is developed using the Django framework with PostgreSQL as the database management system. This chapter presents the important code snippets used in the implementation of the system. The selected snippets demonstrate database models, authentication, URL routing, CRUD operations, form handling, and database interaction.

Only the important sections of code are included to explain the core working of the implemented system. Complete CSS files, configuration files, dependency files, and other supporting code are not reproduced.

5.1 Database Model Definitions

Django models are used to represent database tables. The FoodGo project uses Django's ORM to create and interact with PostgreSQL tables.

5.1.1 Restaurant Model

The Restaurant model stores information about restaurants available in the FoodGo system.

```
class Restaurant(models.Model):

    CATEGORY_CHOICES = [
        ('Pizza', 'Pizza'),
```

```
        ('Burgers', 'Burgers'),
        ('Indian', 'Indian'),
        ('Chinese', 'Chinese'),
        ('Desserts', 'Desserts'),
        ('Healthy', 'Healthy'),
        ('Beverages', 'Beverages'),
    ]

    name = models.CharField(max_length=200)
    description = models.TextField()
    cuisine = models.CharField(max_length=100)
    rating = models.DecimalField(max_digits=2, decimal_places=1)
    delivery_time = models.IntegerField()
    category = models.CharField(
        max_length=50,
        choices=CATEGORY_CHOICES
    )
```

The model is converted into a PostgreSQL table using Django migrations.

5.2 Food Item Model

The FoodItem model stores individual food items available in restaurants.

```
class FoodItem(models.Model):

    restaurant = models.ForeignKey(
        Restaurant,
        on_delete=models.CASCADE
    )

    name = models.CharField(max_length=200)
    description = models.TextField()
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2
    )
```

The foreign key establishes a relationship between a restaurant and its food items.

5.3 Cart Item Model

The CartItem model is used to store food items added to a user's cart.

```
class CartItem(models.Model):

    user = models.ForeignKey(
        User,
        on_delete=models.CASCADE
    )

    food_item = models.ForeignKey(
        FoodItem,
        on_delete=models.CASCADE
    )
```

```
    quantity = models.PositiveIntegerField(default=1)
```

This allows each authenticated user to maintain cart items associated with specific food items.

5.4 User Authentication

FoodGo uses Django's built-in authentication system for registration, login, logout, and protected pages.

5.4.1 Login View

The login view verifies the user's credentials and creates an authenticated session.

```
def login_view(request):

    if request.method == 'POST':

        username = request.POST.get('username')
        password = request.POST.get('password')

        user = authenticate(
            request,
            username=username,
            password=password
        )

        if user is not None:
            login(request, user)
            return redirect('home')

        messages.error(
            request,
            'Invalid username/email or password. Please try again.'
        )

    return render(request, 'accounts/login.html')
```

Django handles password verification using its secure password hashing mechanism.

5.5 Protected Home Page

The home page is protected using Django's `login_required` decorator.

```
@login_required
def home(request):
    return render(
        request,
        'accounts/home.html'
    )
```

Only authenticated users can access the protected page.

5.6 URL Routing

Django URL patterns connect browser requests to the appropriate views.

5.6.1 Restaurant URLs

```
path(
    'restaurants/create/',
    views.restaurant_create_view,
    name='restaurant_create'
),

path(
    'restaurants/update/<int:restaurant_id>/',
    views.restaurant_update_view,
    name='restaurant_update'
),

path(
    'restaurants/delete/<int:restaurant_id>/',
    views.restaurant_delete_view,
    name='restaurant_delete'
),
```

The `<int:restaurant_id>` parameter identifies the particular restaurant that needs to be updated or deleted.

5.7 Restaurant CRUD Operations

The FoodGo project implements CRUD operations for restaurant management.

CRUD stands for:

- **Create** – Add a new restaurant
- **Read** – Display restaurant information
- **Update** – Modify restaurant information
- **Delete** – Remove a restaurant

5.7.1 Create Operation

The create operation accepts restaurant information from a form and saves it to the database.

```
def restaurant_create_view(request):

    if request.method == 'POST':

        restaurant = Restaurant(
            name=request.POST.get('name'),
            description=request.POST.get('description'),
            cuisine=request.POST.get('cuisine'),
            rating=request.POST.get('rating'),
            delivery_time=request.POST.get('delivery_time'),
            category=request.POST.get('category')
        )
```

```
        restaurant.save()

        return redirect('home')

    return render(
        request,
        'accounts/restaurant_form.html'
    )
```

5.7.2 Read Operation

The read operation retrieves restaurant records from PostgreSQL through Django ORM.

```
def restaurant_detail_view(
    request,
    restaurant_id
):

    restaurant = get_object_or_404(
        Restaurant,
        id=restaurant_id
    )

    return render(
        request,
        'accounts/restaurant_detail.html',
        {'restaurant': restaurant}
    )
```

5.7.3 Update Operation

The update operation retrieves an existing restaurant and modifies its information.

```
def restaurant_update_view(
    request,
    restaurant_id
):

    restaurant = get_object_or_404(
        Restaurant,
        id=restaurant_id
    )

    if request.method == 'POST':

        restaurant.name = request.POST.get('name')
        restaurant.description = request.POST.get('description')
        restaurant.cuisine = request.POST.get('cuisine')
        restaurant.rating = request.POST.get('rating')
        restaurant.delivery_time = request.POST.get('delivery_time')
        restaurant.category = request.POST.get('category')

        restaurant.save()

        return redirect('home')

    return render(
        request,
        'accounts/restaurant_form.html',
```

```
        {'restaurant': restaurant}
    )
```

5.7.4 Delete Operation

The delete operation removes a selected restaurant from the database.

```
def restaurant_delete_view(
    request,
    restaurant_id
):

    restaurant = get_object_or_404(
        Restaurant,
        id=restaurant_id
    )

    restaurant.delete()

    return redirect('home')
```

5.8 Food Item CRUD

FoodGo also supports CRUD operations for food items.

5.8.1 Food Item Creation

```
def food_create_view(request):

    if request.method == 'POST':

        FoodItem.objects.create(
            restaurant_id=request.POST.get('restaurant'),
            name=request.POST.get('name'),
            description=request.POST.get('description'),
            price=request.POST.get('price')
        )

        return redirect('foods')

    restaurants = Restaurant.objects.all()

    return render(
        request,
        'accounts/food_form.html',
        {'restaurants': restaurants}
    )
```

The `FoodItem` record is associated with a restaurant through the foreign key relationship.

5.9 Django ORM Database Operations

Django ORM is used instead of writing SQL queries manually for normal CRUD operations.

5.9.1 Selecting Records

```
restaurants = Restaurant.objects.all()
```

This retrieves all restaurant records from the database.

5.9.2 Creating a Record

```
Restaurant.objects.create(
    name='Flame & Crust Artisan Pizzeria',
    description='Fresh and delicious pizzas',
    cuisine='Italian',
    rating=4.5,
    delivery_time=30,
    category='Pizza'
)
```

5.9.3 Updating a Record

```
restaurant = Restaurant.objects.get(id=1)

restaurant.name = 'Updated Restaurant'
restaurant.rating = 4.7

restaurant.save()
```

tirestaurant `= Restaurant.objects.get(id=1)`

```
restaurant.delete()
```

5.10 Django Migration

Django migrations are used to apply model changes to the PostgreSQL database.

```
python manage.py makemigrations
python manage.py migrate
```

The `makemigrations` command creates migration files based on changes made to the models, while `migrate` applies those changes to the PostgreSQL database.

5.11 Complete CRUD Workflow

The CRUD workflow of the FoodGo system can be summarized as follows:

1. The user opens the FoodGo web application in the browser.
2. The user logs into the system using Django authentication.
3. The user accesses the restaurant or food management section.
4. A form collects the required restaurant or food item information.
5. Django receives the HTTP request through the configured URL.
6. The corresponding Django view processes the request.
7. Django ORM performs the required database operation.
8. PostgreSQL stores, retrieves, updates, or deletes the data.
9. Django returns an HTTP response and displays the updated information on the webpage.

5.12 Why Only Important Code Is Included

The complete FoodGo project contains additional HTML templates, CSS styling, Django configuration, migration files, authentication templates, static files, and supporting code.

Including every source file in the project report would make the report unnecessarily large. Therefore, this chapter includes only the important code snippets that demonstrate the core implementation of the FoodGo system.

The selected snippets cover:

- Django database models
- PostgreSQL database integration
- User authentication
- URL routing
- Restaurant CRUD operations
- Food item CRUD operations
- Cart item model
- Django ORM operations
- Database migrations
- Complete CRUD workflow

These snippets provide an overview of how the FoodGo application communicates with the PostgreSQL database and implements its major functionality.

Chapter 6: Result

6.1 Login Page Output

The login page is the entry point of the FoodGo application. It allows registered users to securely access the system using their username and password.

The page contains the FoodGo branding, username/email input field, password field, and login button. An option for new users to register is also provided.

The login system is implemented using Django's built-in authentication framework. If valid credentials are entered, the user is redirected to the FoodGo home page. If invalid credentials are entered, an appropriate error message is displayed.

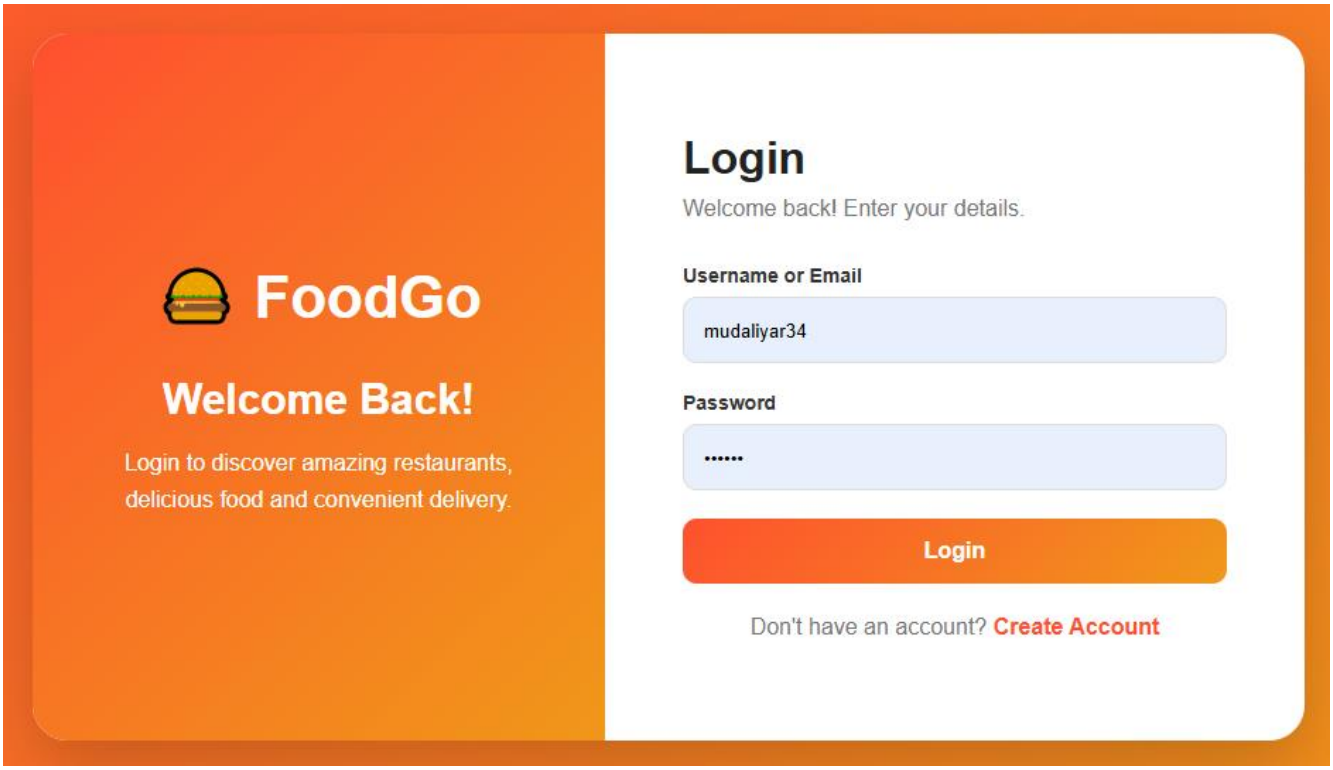


Figure 6.1: Login Page Output

6.2 Admin Dashboard Output

After administrator authentication, the Django Admin Dashboard is displayed. The FoodGo administration panel provides access to the application's database records and management functionality.

The administrator can manage users and application data through the Django administration interface. The dashboard provides navigation to the available database models and allows authorized administrators to add, view, modify, and delete records.

For the FoodGo system, the admin panel can be used to manage data such as users, restaurants, food items, and cart items.

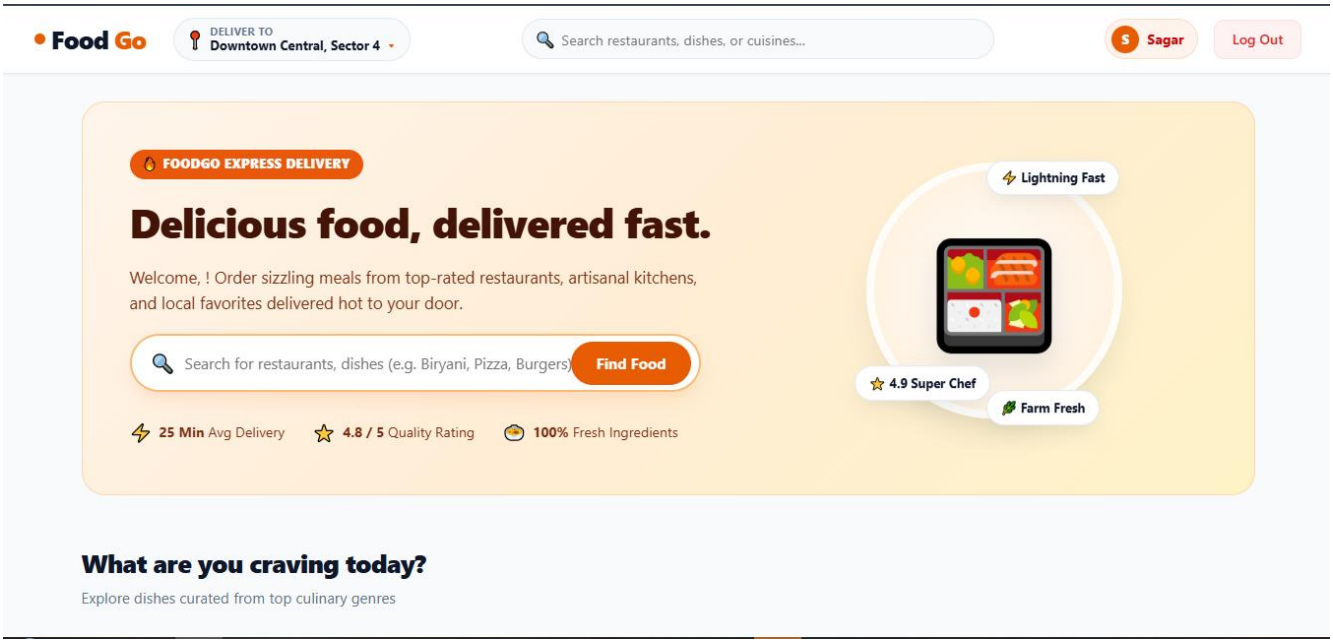


Figure 6.2: Admin Dashboard Output

6.3 Restaurant Management Output

The Restaurant Management page is the main administrative CRUD screen of the FoodGo application. It allows the administrator to manage restaurant records stored in the PostgreSQL database.

The page provides an option to add a new restaurant and displays the existing restaurant records. Each restaurant record contains information such as restaurant name, description, cuisine, rating, delivery time, and category.

The administrator can perform the following CRUD operations:

- **Create:** Add a new restaurant to the system.
- **Read:** View the list of available restaurants.
- **Update:** Edit the details of an existing restaurant.
- **Delete:** Remove a restaurant from the system.

The restaurant records displayed on the page are retrieved from PostgreSQL using Django ORM

FoodGo

Restaurant Management

Add Restaurant

Name:

Burger king

Description:

Best burger

Cuisine:

Burgers

Rating:

4.9

Delivery time:

20–30 min

Price for two:

350

Image:

Choose File No file chosen

Category:

Pizza

Is active:

Save Restaurant

← Back

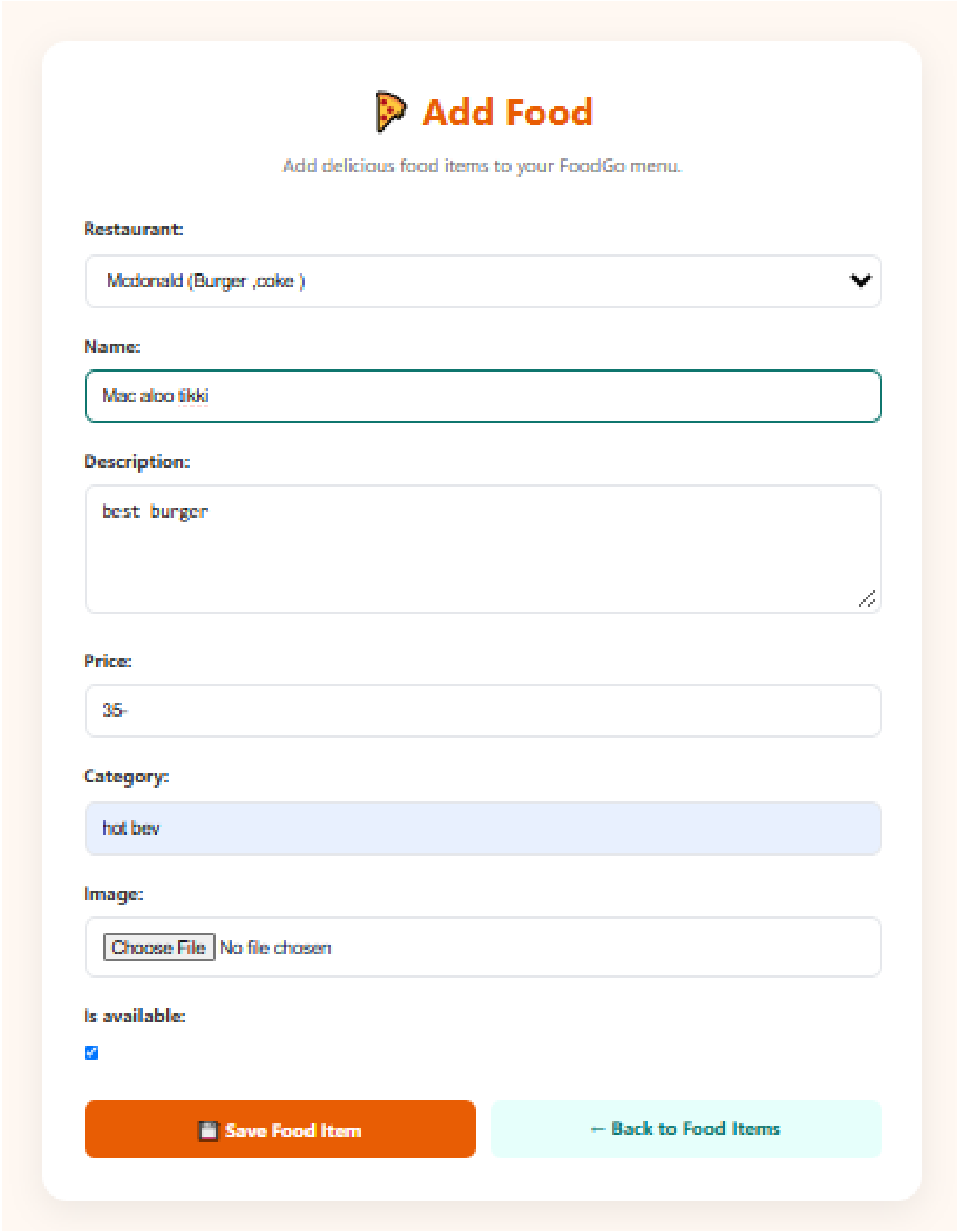


Figure 6.3: Restaurant and Foodt CRUD Output

6.4 Food Item CRUD Result

The Food Item Management module was tested during development. Food items were added to the database, displayed on the FoodGo interface, edited, and deleted successfully.

Operation	Test Action	Result
Create	Add a new food item	Passed
Read	Display food items	Passed
Update	Change an existing food item's information	Passed
Delete	Delete the selected food item	Passed

The successful completion of these operations confirms that the Django application is communicating correctly with the PostgreSQL database for food-item management.

6.5 Database Output

The FoodGo project uses PostgreSQL as its relational database management system. The database stores application data including user information, restaurants, food items, and other implemented project entities.

Django ORM is used to communicate between the application and PostgreSQL. Database migrations were created and applied during development to keep the database structure synchronized with the Django models.

The PostgreSQL database used by the project is named food_delivery.

6.6 CRUD Result Summary

The implemented CRUD functionality was tested through the FoodGo web interface.

Module	Create	Read	Update	Delete
Restaurants	Passed	Passed	Passed	Passed
Food Items	Passed	Passed	Passed	Passed

The CRUD operations demonstrate that records can be inserted, retrieved, modified, and removed through the application's web interface while the changes are persisted in PostgreSQL.

6.7 Result Summary

The major results achieved during the development of FoodGo are:

• User registration and login functionality is working.

• Login can be performed using username or email.

• Logout functionality is implemented.

• Protected pages are available to authenticated users.

• PostgreSQL connectivity has been established.

• Restaurant management CRUD functionality has been implemented.

• Food Item CRUD functionality has been implemented.

• Django ORM is used for database operations.

• Database migrations have been created and applied.

• Restaurant and food records can be managed through the web interface.

• The FoodGo homepage provides the main interface for the application.

CHAPTER 7: CONCLUSION

7.1 Conclusion

The FoodGo project successfully establishes a web-based food-delivery management application using Django and PostgreSQL. The project provides a foundation for managing users, restaurants, and food items through a single web application.

The development process included configuring the Django environment, connecting Django with PostgreSQL, implementing user authentication, creating database models, applying migrations, and developing CRUD operations for restaurants and food items.

The authentication system provides registration, login, logout, and protected-page functionality. Django's built-in authentication and password-hashing mechanisms were used to provide secure user authentication.

The Restaurant Management module demonstrates the complete CRUD lifecycle. The Create operation adds a restaurant, the Read operation displays restaurant information, the Update operation modifies existing information, and the Delete operation removes a selected restaurant.

Similarly, the Food Item Management module provides complete CRUD functionality. Food items can be added, displayed, updated, and deleted through the application.

The project demonstrates how Django can be integrated with PostgreSQL to create a database-driven web application. The use of Django models, migrations, views, URLs, templates, and PostgreSQL provides a structured foundation for further development of the FoodGo platform.

7.2 Challenges Encountered During Development

The following challenges were encountered during the development of the FoodGo project:

• Configuring the Django development environment on Windows.

• Connecting Django to the PostgreSQL database.

• Installing and configuring the PostgreSQL database driver.

• Understanding Django project and application structure.

• Creating Django models for restaurants and food items.

• Creating and applying database migrations.

• Connecting URLs with the correct Django views.

• Implementing authentication using Django's built-in authentication system.

• Handling login using both username and email.

• Implementing CRUD operations for restaurants.

• Implementing CRUD operations for food items.

• Correctly connecting templates, views, models, and URLs.

• Debugging Django application errors during development.

• Testing database changes through the web interface.

7.3 Learning Outcomes

The FoodGo project provided practical understanding of the following concepts:

• Understanding Django project and application structure.

• Understanding Django's Model-View-Template architecture.

• Creating database models using Django ORM.

• Connecting Django with PostgreSQL.

• Creating and applying database migrations.

• Performing database CRUD operations.

• Implementing user registration and authentication.

• Understanding password hashing and authentication.

• Creating Django views and URL patterns.

• Connecting HTML templates with Django views.

• Displaying database records dynamically.

• Updating and deleting database records.

• Debugging Django application errors.

• Understanding relational database concepts.

• Integrating frontend templates with backend database operations.

7.4 Future Scope

The current FoodGo project provides a foundation that can be expanded with additional food-delivery features. Future development may include:

• Implementing a complete shopping cart system.

• Adding quantity management for cart items.

• Implementing order management.

• Adding checkout functionality.

• Adding payment integration.

• Implementing order tracking.

• Adding delivery-status management.

• Adding restaurant search and filtering.

• Adding food-category filtering.

• Adding restaurant reviews and ratings.

• Adding user profile management.

• Adding restaurant-owner or administrator dashboards.

• Adding improved validation and error handling.

• Adding REST APIs for mobile or external applications.

• Deploying the application to a production server.

7.5 Final Project Vision

The final vision of FoodGo is to provide a complete web-based food-delivery platform where users can discover restaurants, browse food items, add items to a cart, place orders, and track their deliveries.

The current implementation provides the foundation for this vision through user authentication, PostgreSQL database connectivity, Restaurant Management CRUD, Food Item Management CRUD, and the FoodGo homepage.

Future modules such as cart management, orders, payments, and delivery tracking can be integrated with the existing database-driven architecture to develop FoodGo into a complete food-delivery application.

CHAPTER 8: REFERENCES

• Django Documentation – https://docs.djangoproject.com/

• PostgreSQL Documentation – https://www.postgresql.org/docs/

• Python Documentation – https://docs.python.org/

• Django Authentication Documentation – https://docs.djangoproject.com/en/stable/topics/auth/

• Django Models Documentation – https://docs.djangoproject.com/en/stable/topics/db/models/

• Django Migrations Documentation – https://docs.djangoproject.com/en/stable/topics/migrations/

• PostgreSQL Database Management System used for the FoodGo project.

• FoodGo project source code developed during the project.

• FoodGo database models and PostgreSQL database implementation.

Note: The report documents only the functionality implemented and tested during the project. The completed functionality includes user authentication, PostgreSQL database connectivity, Restaurant Management CRUD, Food Item CRUD, and the FoodGo homepage. Features such as payments, delivery tracking, and complete order processing are considered future enhancements unless separately implemented and tested.